Lecture 13: Practical advice

Max G'Sell

Machine Learning I: 46-926

December 16, 2020

Announcements

- We are done with homeworks!
- There will be one more quiz, coming out by Friday morning and open through December 21.
- For grade concerns: As mentioned previously, I curve quizzes/final scores upward as necessary when determining final grades. I don't curve downward.

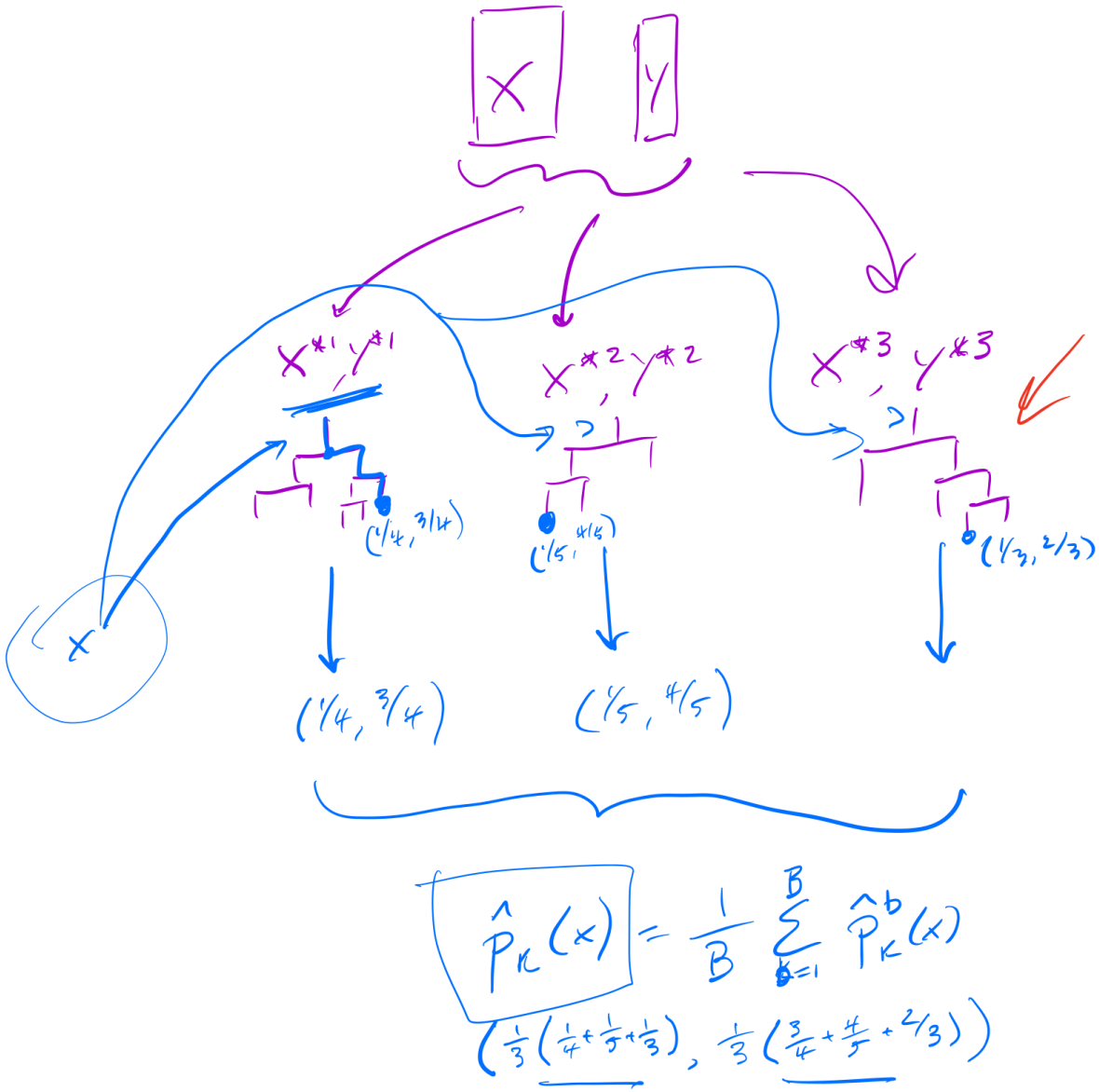
Today

Today we will say a few final things about random forests, then move on to practical advice about machine learning.

There's a lot that I'd like to say today, so I'm going to mostly save quiz/homework discussion until the end. If it comes to it, I'll discuss them after the end of lecture and you can watch the video one day when schedules are calmer.

- Examining random forests
- Practical advice
- Homework and quiz discussion

Bagging


X
y
X^{*1}, y^{*1}
X^{*2}, y^{*2}
X^{*3}, y^{*3}
(1/4, 3/4)
(1/5, 4/5)
(1/3, 2/3)
x
(1/4, 3/4)
(1/5, 4/5)
$\hat{p}_k(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{p}_k^b(x)$
$\left(\frac{1}{3}\left(\frac{1}{4}+\frac{1}{5}+\frac{1}{3}\right), \frac{1}{3}\left(\frac{3}{4}+\frac{4}{5}+\frac{2}{3}\right)\right)$

Random forest properties

Random forests have some amazing properties, especially when compared to other methods we've looked at.

- Surprisingly impressive predictive performance!
- Automatic handling of variable transformations!
- Automatic detection of interactions!
- What about Interpretability?

Digging into Random Forests

The bagged classifier is now an average of many trees. This is much harder to interpret! That was half the reason we loved trees!

Similarly, one of the big selling points of the Lasso is that it produces *interpretable* models. By seeing the sparsity pattern, we know which variables are being used for predicting.

We see that random forests predict incredibly well. However, an average of hundreds of random trees is harder to understand.

We need tools to peek into these "black box" methods to understand how they're using the data.

Variable Importance

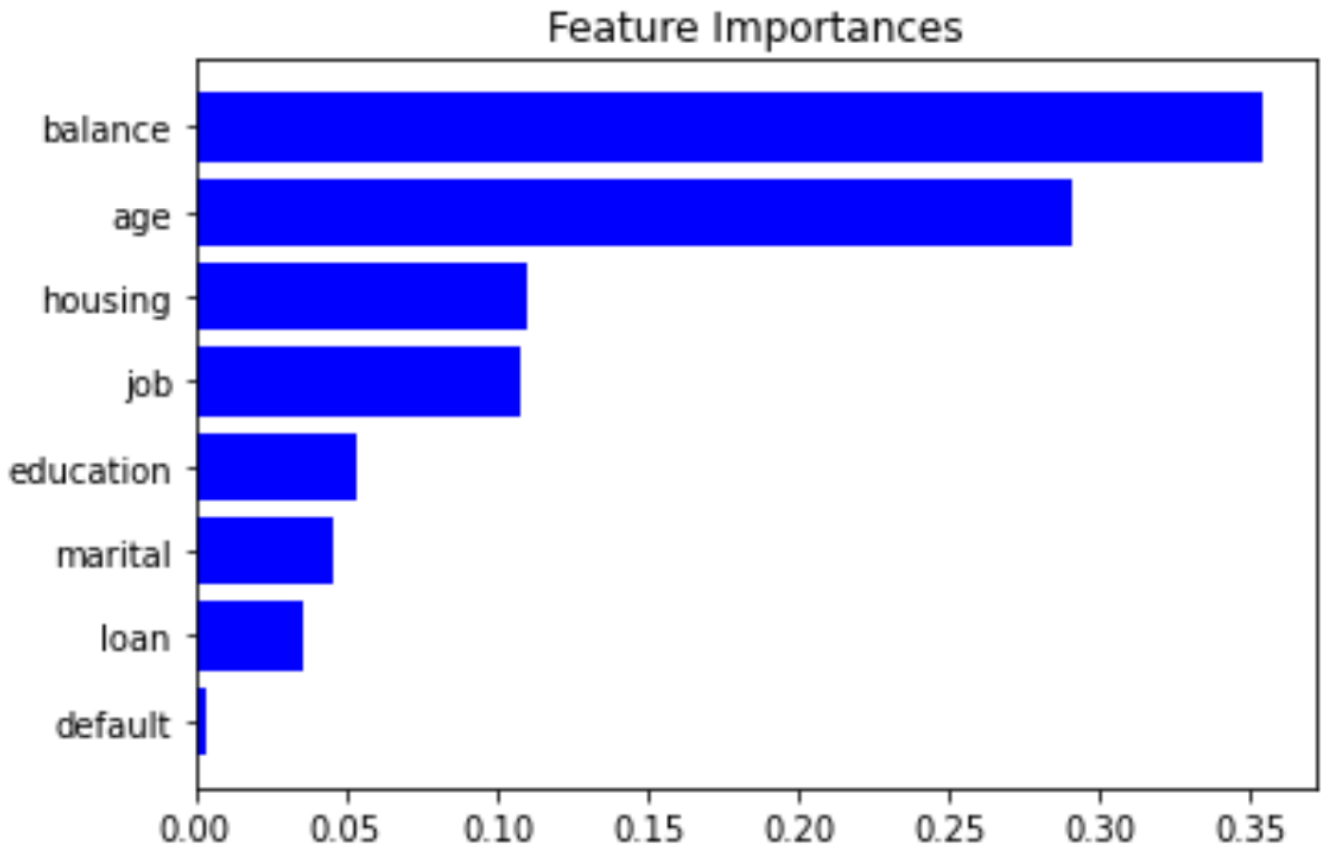
The most basic task is to understand which predictors are important for making the random forest prediction.

Random forests don't explicitly do variable selection like the lasso, but their individual trees tend to rely more on informative variables when they search for the next split.

There are two popular ways to measure variable importance:

1. For each variable, measure the amount that RSS (or Gini index) decreases due to splits in that variable. Average this over all trees in the forest
2. Randomly permute each variable (one at a time) and see how much the model performance decreases.

The first of these is what sklearn computes for feature importance.


Feature Importances
balance
age
housing
job
education
marital
loan
default
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35

Partial dependence plots

Variable importance tells you which variables are useful, but not how they are used.

Partial dependence plots can give some insight into the way that estimates change with each variable.

Suppose we divide our variables into sets $\mathcal{S}$ and $\mathcal{C}$, and we want to assess the effect of the variables in $\mathcal{S}$ on our predictions.

The partial dependence of $f(X)$ on $X_{\mathcal{S}}$ is defined as

$$f_{\mathcal{S}}(X_{\mathcal{S}}) = \mathbb{E}_{X_{\mathcal{C}}} f(X_{\mathcal{S}}, X_{\mathcal{C}})$$

This can be estimated by

$$\bar{f}(X_{\mathcal{S}}) =$$

where $x_{i\mathcal{C}}$ are just the values that appear in our data.

Partial dependence

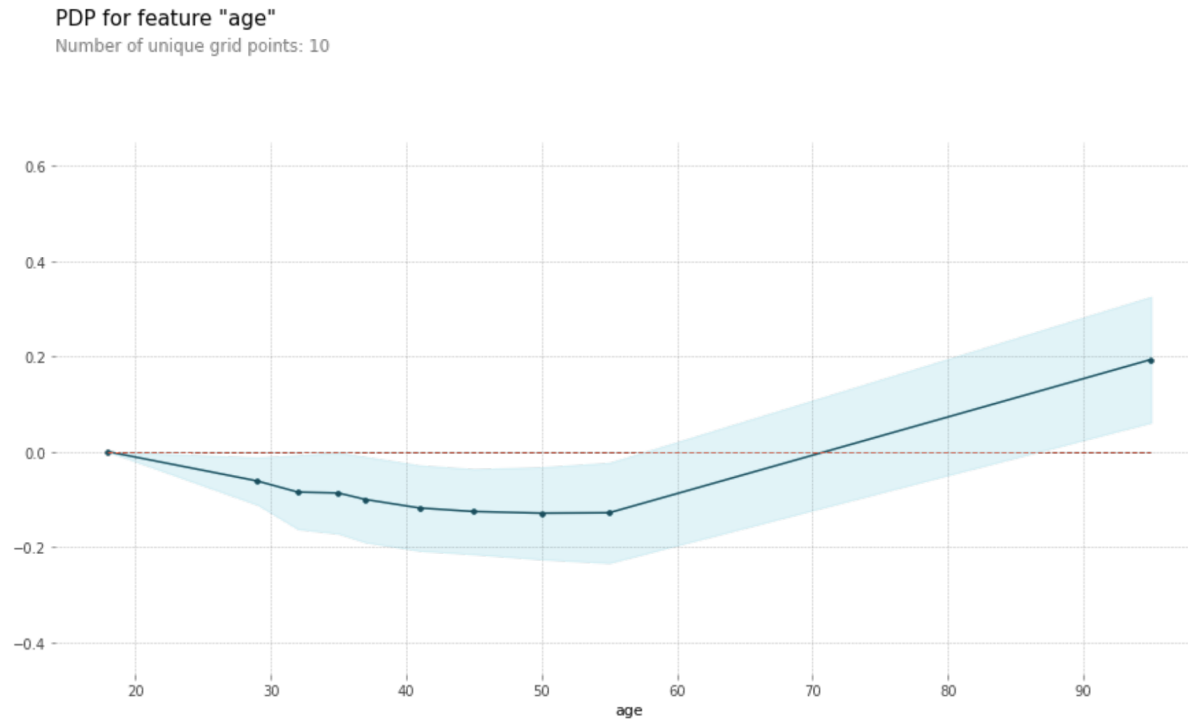
Partial dependence works out to be a nice definition for simple models.

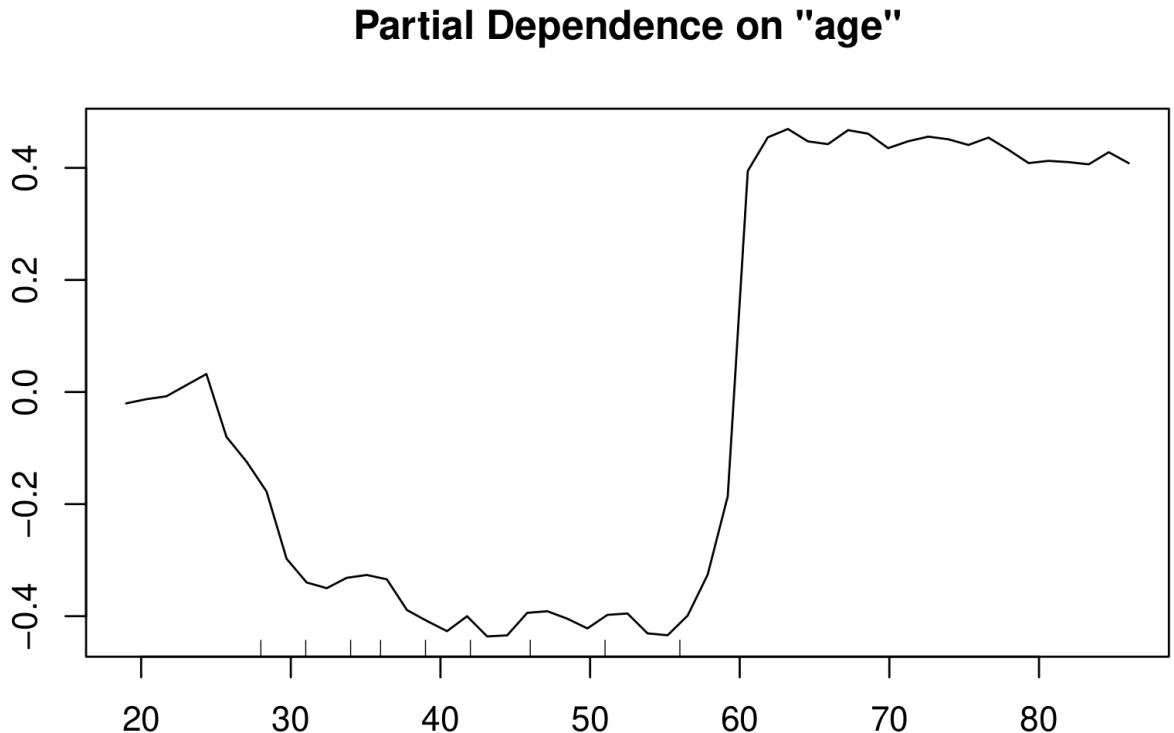
Suppose the truth is that $f(X) = f_1(X_1) + f_2(X_2)$. Then

$$\bar{f}(X_1) = \mathbb{E}_{X_2} f(X_1, X_2) = \mathbb{E}_{X_2} \left(f_1(X_1) + f_2(X_2)\right)$$
$$= $$

Suppose the truth is that $f(X) = f_1(X_1) \cdot f_2(X_2)$. Then

$$\bar{f}(X_1) = \mathbb{E}_{X_2} f(X_1, X_2) = \mathbb{E}_{X_2} \left(f_1(X_1) \cdot f_2(X_2)\right)$$
$$= $$


PDP for feature "age"
Number of unique grid points: 10
0.6
0.4
0.2
0.0
−0.2
−0.4
20
30
40
50
60
70
80
90
age


Partial Dependence on "age"
0.4
0.2
0.0
−0.2
−0.4
20
30
40
50
60
70
80

Disadvantages

It is important to discuss some disadvantages of bagging:

- *Loss of interpretability:* the final bagged classifier is not a tree, and so we forfeit the clear interpretative ability of a decision tree

- *Computational complexity:* we are essentially multiplying the work of growing a single tree by B. We similarly multiply the cost of producing a new prediction.

  (Do note that these operations are both naively parallelizable.)

Random forests

Random forests wind up being one of the best all-around tools for prediction, outside of highly structured tasks where deep learning tends to dominate.

Random forests automatically perform variable selection (and transformations!), making them scale well with dimension. In the same way, they automatically detect important interactions, avoiding the need to recognize and specify them explicitly.

The variables and interactions that the random forest chooses to use can even guide later method tuning and development.

Random forests

While random forest tuning parameters do matter, simple defaults work quite well. Most importantly, it seems difficult to accidentally overfit random forests as badly as other high-performance methods. Most important for tuning:

- Have enough trees.
- Pick a reasonable depth or min leaf/split size. (They capture about the same thing. Pretty redundant.)
- Think about class balance, weights, sampling.

Boosting (ML2) can sometimes achieve better performance, but only at the cost of additional tuning and a much higher risk of overfitting.

Practical thoughts

You have data, now what?

Perhaps someone gives you data. In more realistic circumstances, you've just wrestled with databases, merging together your own data set. Now what?

Note: there's necessarily an aspect of opinion in some of this, since there are many ways to approach new problems.

Getting started

What is my data?

- Variables, values, size
- Issues: Missingness, outliers, balance
- Distributions, relationships, patterns

What problem am I actually trying to solve?

- Loss functions
- Constraints

How am I actually going to validate and protect myself?

- Test sets, cross-validation

Get a feel for what's in your data set

Just explore your data set, so you know what to expect

- What is one observation, and does this make sense?
  - How should I think of multiple observations on the same entity? (Panel data)
- Dimension of the data, variables and obervations.
- Variable types: factors, dates, numeric.
- Variable distributions: Number of factor levels, imbalance of factor levels, skewness of variables.
- Simple relationships between variables.

Plot your data!

You will have seen plenty of this in Data Science, so we won't get into it much, but will give some ideas. `ggplot` and related libraries are excellent for this.

Ideas:

- Plot univariate and bivariate distributions of variables. Are variables continuous? Bump like? Mixture like?
- Look at conditional plots and investigate differences
- Is there temporal or spatial structure? Plot there.

This will:

- Inform your modeling
- Guide your feature extraction

Outliers

Outliers are not very well-defined, so there's no one method for finding them.

You are worried about points that are abnormal enough that they unduly influence your model. They may represent:

- Mistakes
- Observations that are not coming from the process you want to model

Points that are not helpful to think of as part of $\mathcal{P}$.

They can be identified in plots, or by a variety of rules (large Mahalanobis distance, high leverage, and many more).

Sometimes you can fix them (typos, special events). Other times this entry is essentially missing data.

Missing data

You will often find missing values in data sets. You need to decide how to handle them when you make predictions.

- You could remove these whole observations. If the missingness is not *informative*, this only costs you data. If it is informative, you are also losing information and biasing your population.

- You could try to fill in the missing values: imputation
  1. Use a strongly correlated variable to predict this one.
  2. Nearest neighbors: find the K closest other points and average their value for this entry.

  Careful! Is your training data reflective of the missing data??

- You could code the NA information as another value

- You could use a method that is robust to missing values:

What is your outcome of interest?

Is it a continuous regression problem?

- Do you really care about the value, or just whether it's big?
- How is your variable behaved? Heavily skewed? Do you see many examples of the big values that you care about?

Is it a classification problem?

- How many classes do you have?
- What are the default class proportions? What is your base rate? Do you have severe imbalance?
- What is your goal and what are your costs?
  - Do you want better misclassification, or just enriched subsets of the data?
  - Are your costs asymmetric enough that you should be thinking about different cutoffs, sensitivity/specificity, etc?

Is it neither regression or classification??

Maybe you're really trying to:

- Find groups of things
- Identify patterns of behavior
- Predict the time until the next event
- Rank items
- Make online decisions
- Learn a strategy without immediate feedback

Constraints on my classifier

Are there practical constraints on my classifier?

- Is my training time severely limited? (Pretty rare)
- Is my evaluation/prediction time severely limited? (More common)
  - Maybe I can only afford to evaluate linear classifications, and random forests are too expensive
- Maybe I'm trying to construct a portfolio. This may have to be linear, or even more constrained (linear and integer, for example)
- Does my model need to be interpretable?

Note: even if my final classifier must be simple (e.g., linear), I can still learn a lot from more flexible models.

Start learning

Now you're set up for your learning task. What are the first things to think about?

- Validation! You want to plan this workflow from the start, before you lie to yourself
- Features! Extract reasonable first set of features to use for prediction (an art, especially text/timeseries)
- What to use? I like to start with something simple and interpretable, and something flexible.

  Another way to think about it: Something that's good in high signal/noise and something that's good in low signal/noise.

Validation

If we magically knew exactly the right model and how well it worked, we could just fit our model on all n points.

In reality, we need "extra" data to:

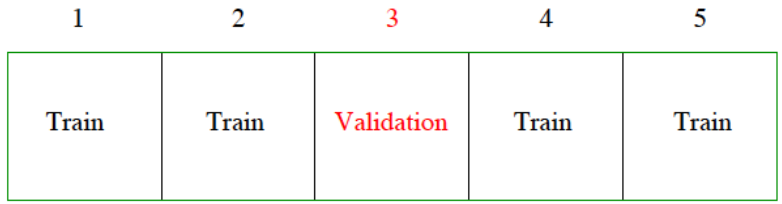
- Determine the appropriate model to fit
- Estimate the chosen model's performance on new data

Work flows

There are several possible data splitting work flows. Here's one reasonable approach.

Cross-validation

One approach to avoid splitting too much is cross-validation. This is an efficient way to select our model without wasting too much data.



However, there is still some concern of overfitting on the cross-validation set. This is especially an issue if we experiment with many models. Thus, a safe and reliable approach is to do both!

Validation plan

A realistic approach:

1. Split the data into a training set and a testing set.
2. Do all your model selection and experimentation on the training set. Use cross-validation or out-of-bag errors only on this set to pick your model.
3. Fit your chosen model on the whole training set.
4. At the very end, use your testing set to get an accurate picture of how well you actually do with that model.
5. For production, go back and fit that model on the whole data set.

You never touch the testing set during your model experimentation and selection.

Some warnings about cross-validation

Cross-validation seems like a magical solution to an impossible problem. There are a few things to beware of:

1. Can have a high computational cost, especially as the number of parameters/models grows

2. When the number of models is very large, it can give misleading results. It is not immune from the usual problems with multiple testing.

3. The folds need to include the entire estimation pipeline. A frequent mistake is to carry out part of the model selection before breaking the data into folds, which invalidates the result.

Warning: high computational cost

Suppose it costs c to fit an estimator once.

To fit it across a grid of 100 λ values, it would cost $100c$.

If we do 10-fold cross-validation across this grid, it would cost $1000c$.

If we do LOOCV, it would cost $100n \cdot c$, which could potentially be tremendously large

If we decide to tune two parameters with 10-fold cross-validation (each with a 100-value grid), it would cost $100000c \ldots$

We want to be somewhat judicious in what we choose to tune with CV (especially given the next slide). There is also a literature in statistics and computer science which works on tricks to avoid these high costs.

Warning: high number of models

If we search a very large number of models, one of them may look better completely by chance.

Imagine we are in a binary classification problem ($y_i \in \{0, 1\}$). If each model were terrible and just flipped coins to make the test set prediction, eventually we would try one that looked quite good. In particular, we would eventually find one that performed better in cross-validation error than the true best model!

This is just something to be aware of when the set of possible models is expanded dramatically. It is less of a risk in more constrained settings

Warning: excluding part of the pipeline

This mistake happens a lot, and looks slightly different each time.

Remember: For cross validation (or test sets) to work, every data-driven decision about the model has to exclude the held-out fold (or the test set).

Bad CV example: Suppose that we observe y and $x_1, \ldots, x_p \in \mathbb{R}^n$, with $p \gg n$. We would like to test our magical prediction algorithm $\hat{f}_\lambda$. Because p is so large, we first filter for x_i which have some reasonable correlation with y, building $\tilde{X}$ with columns $\{x_i : |\mathrm{cor}(x_i, y)| > \delta\}$.

Now we estimate $\hat{f}_\lambda$ on $\tilde{X}, y$ and use cross validation to tune λ and estimate the error.

What goes wrong??

Cross-validating Time series

Cross-validating time series often takes special care, since the assumption that $(X, Y) \overset{iid}{\sim} \mathcal{P}$ is usually violated in practice. This means that we can't just randomly shuffle data. We would wind up predicting the past having seen part of the future!

There are many different approaches to cross-validating time series. What is appropriate depends on how you're going to be rolling out your predictor in practice. For example:

1. You might be training a model at every instant and evaluating it on the next time point.

2. You might be training a model occasionally (every month? week?) and then running it until the next update.

Feature generation

The general lesson that has been learned through many applications and contests is that the data and features are much more important than the exact algorithm that you apply.

Tweaking the algorithm gives small gains. Developing new features to feed into your algorithms gives big gains.

This is where the power of forests, boosting, and deep learning come in. They have an ability to use features more flexibly (forests/boosting at a very basic level).

Feature generation

It is difficult to give much specific advice about feature generation, since it is so problem-dependent

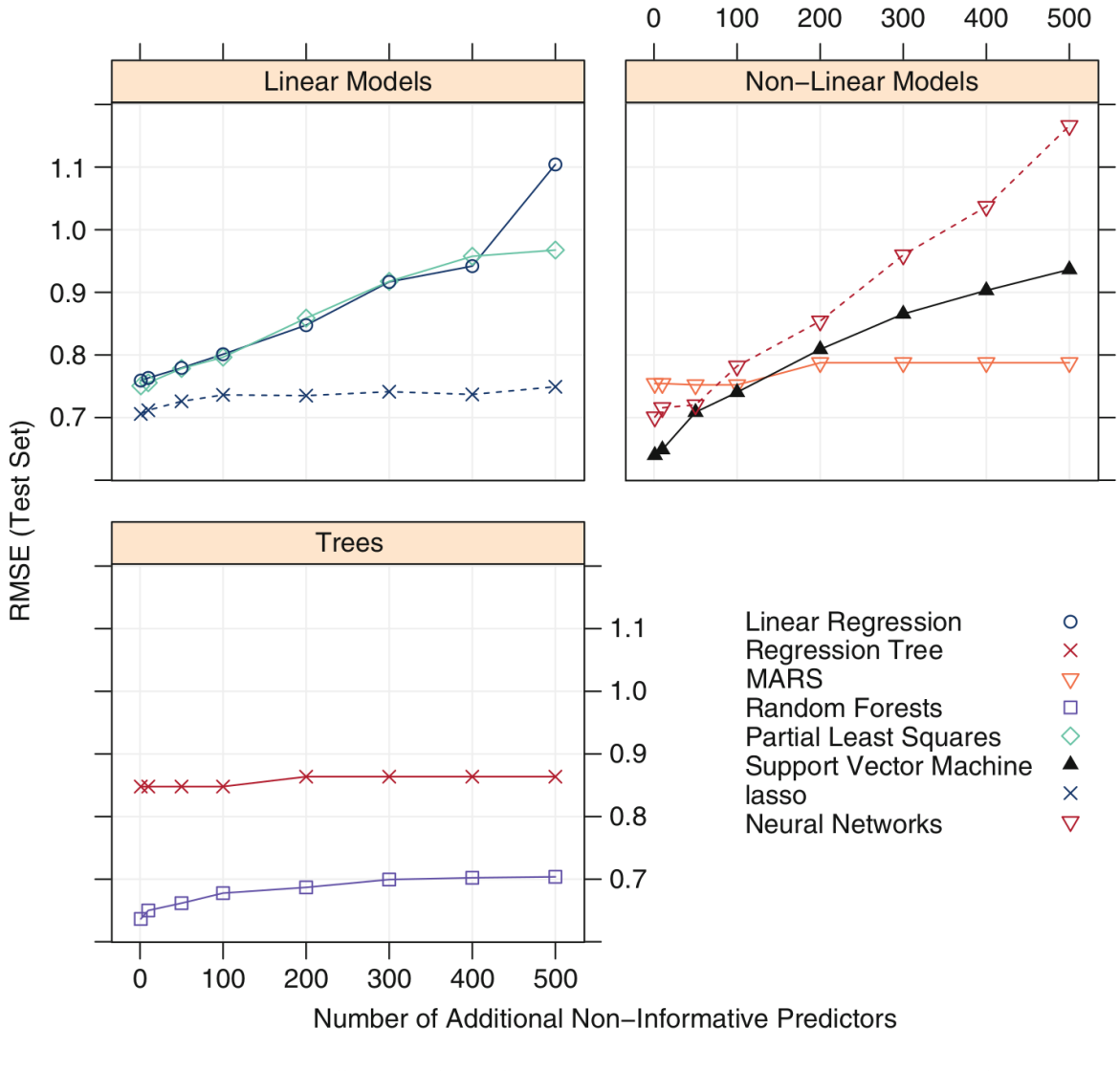
A few general comments:

- With modern model selection algorithms (e.g., Lasso, random forests), having extra features does not hurt much. You can afford to generate many useless features in pursuit of a few really good ones.

  This means you can consider multiple transformations or versions of the same information. Especially useful with text/time series data.

- Make sure only to use information you would have in new samples! For example: you can't use signs that a company has gone out of business when predicting its viability. (This happens...)

With modern algorithms, you can afford to generate many useless features while pursuing really good ones.



(APM 19.1)

Features for time series

Common features for time series include:

- Simple summaries of each series: Mean, standard deviation, skewness, slope, last value, max/min value.
- First differences of the series, integrals of the series (and then simple features again).
- Features of the wavelet or fourier transforms. Features of fitted ARIMA-style models, sometimes.

Beautiful example: `http://web.stanford.edu/~lmackey/papers/alsprize4life-slides.pdf` slides 21-32

Time itself

You'll often have time stamps or dates. First, these are often annoyingly formatted, so you need to make them load correctly.

You also often want to featurize these in a way that your algorithm can make use of extra information. Possibly extract additional features:

- Work hours
- Work day
- Month, quarter, season
- Holiday
- Important announcements/events

First models

Personally, I like to start with:

- Something simple and interpretable. Often Lasso (prediction) or Logistic Lasso (classification).
- Something flexible and more powerful. Often random forests.

Differences between the two models can hint at what you might be ignoring with the simple model. You can explore the difference and try to see why.

Variable importance and partial dependence plots from the random forest can suggest important variables, new transformations, and important interactions.

Sidenote: Don't do this!

Everyone loves the LASSO, and everyone loves Random Forests...

A result of this is that many people will... fit the lasso, take the selected variables, and then fit a random forest on that subset of variables.

Don't Do This.

The random forest doesn't need this help with variable selection, it scales fine. Furthermore, you're just throwing away the non-linear relationships and interactions that random forests are designed to pick up for you!

Now what?

You have your data. You have some features. You've fit a couple models and have estimates of their error.

Now what? How do you improve your error?

(Also, what error should you actually care about?)

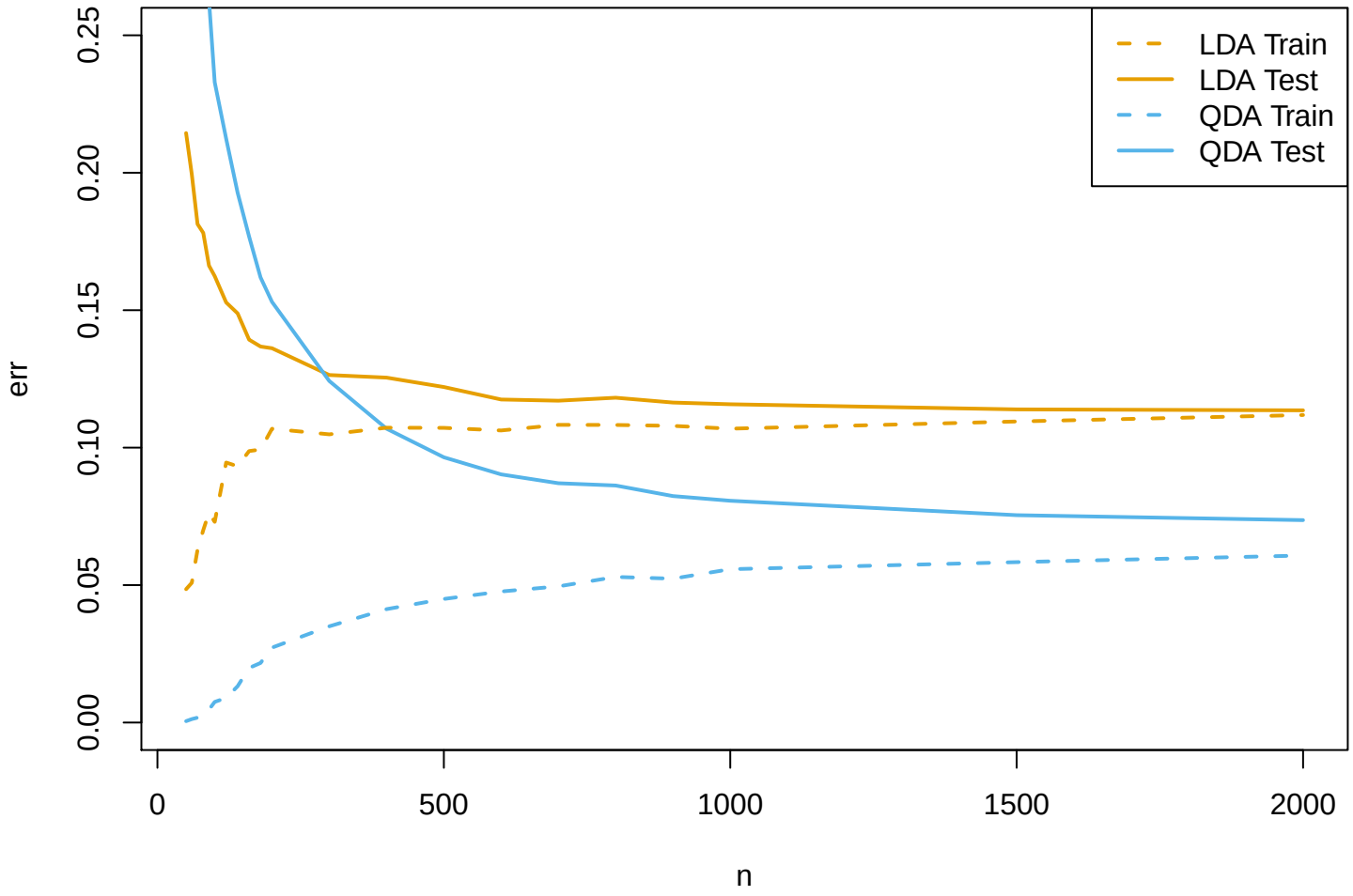
One approach: Think about Bias-Variance Tradeoff! Remember, all the error you can correct is either in your bias or your variance.

Is your current model too biased or too variable?

One way to get a hint: look at the difference between training and test error.

What does it mean if:

- The training error is much smaller than the test (CV) error?
- The training error is about the same as the test (CV) error?



LDA (less flexible) with fixed p and varying n. True model fits QDA (more flexible) assumptions, but it can be too high variance.

What can you do to change your model? What do these do to the bias and variance?

- Get more data:
- Make more/better features:
- Use a more flexible model:
- Use a more regularized model:

Tweaking features

You can look at the variables that are currently important to give some hints of better variables.

If a variable is important, how can you make it more useful? What other variables might it suggest are useful?

If it is not important but you thought it should be, what is going wrong?

Variable importance plots can hint at useful variables when included with transformations or interactions. Partial dependence plots can reveal better transformations and even interactions.

What errors do you care about?

In regression problems: Is least squares error reasonable for your problem?

In classification: Is misclassification error appropriate? Do you care about false positives and false negatives equally? How should you trade them off?

Correcting for badly imbalanced samples

When one class is much smaller than the other, som of our classifiers don't do very well. There are a few approaches to fixing this:

Suppose that you have 90% class 1, and 10% class 2.

- Downsampling: Sample one (or a few) elements of class 1 for each element of class 2 to make it more balanced
- Upsampling: Duplicate elements of class 2 to make it more balanced.
- Artificially change your prior weights, class weights, or case weights. This depends a bit on which method you are using.

Always keep an eye on your base rate: the fraction of each class in your data. That keeps you aware of this problem, and lets you know what good performance really is: 10% misclassification in the problem above is not very impressive...

Misclassification rate and imbalanced data

We know that we should classify as $Y = 1$ when $P(Y = 1|X = x) > 0.5$ to minimize expected misclassification rate. Suppose that an event only happens 5% of the time in the general population.

By Bayes Theorem,

$$P(Y = 1|X = x) = P(Y = 1)\frac{P(X = x|Y = 1)}{P(X = x)}$$

Therefore, to classify as $Y = 1$, we would need to see an X that is 10 times more likely in the $Y = 1$ population than the overall population.

Just because rebalancing can build a better classifier, it might not be enough to truly flip our classifications!

Ensemble Learning

As a reminder, if you've spent a lot of time building different models, you might think about ensembles of models.

We talked about a few different versions of ensembling:

- Voting, weighted voting
- Averaging
- Stacking: Using their guesses as input to another classifier!

These approaches can lead to some improved performance, at the cost of interpretability, fragility, and potential difficulty tweaking later.

Further resources

A very nice discussion of tuning advice can be found in the slides from Andrew Ng's machine learning class:
`http://see.stanford.edu/materials/aimlcs229/ML-advice.pdf`

What's next?

Going into ML2, we have a few classic supervised topics left:

- Boosting
- Support Vector Machines (SVMs)
- Generative models: LDA, Naive Bayes

Then we will move on to special topics like:

- Revisiting clustering and PCA
- Mixture models, matrix factorization, topic models, MCMC (probably)
- Waiting time prediction - Survival analysis
- Deep learning
- Reinforcement learning

I will be away next semester. Prof. Alex Reinhart will be taking over for ML2.